

PL/SQL

II1

2025/2026

hela.boukef@ensi-uma.tn

Plan

- Blocs PL/SQL : structure
- Variables
- Traitements conditionnels
- Traitements répétitifs
- Curseurs
- Exceptions
- Procédures et fonctions
- Triggers

Structure du bloc PL/SQL (1/11)

Le langage **Oracle PL SQL** est :

- un langage procédural
 - une extension procédurale du langage SQL
-
- Il permet de grouper des traitements et de les soumettre au noyau en un bloc unique de traitement.
 - Le langage SQL est non procédural alors que le **PL SQL** est un langage procédural. Le **PL SQL** sert à programmer des procédures, des fonctions, des triggers, des packages.
 - Des variables permettent l'échange d'information entre les requêtes SQL et le reste du programme

Structure du bloc PL/SQL (2/11)

Supposons que nous voulons accorder un bonus à chaque employé en fonction des heures effectuées sur un projet.

Le problème serait simplifié si vous disposiez d'instructions conditionnelles.

→ Le langage PL/SQL est conçu pour faire face à ces besoins

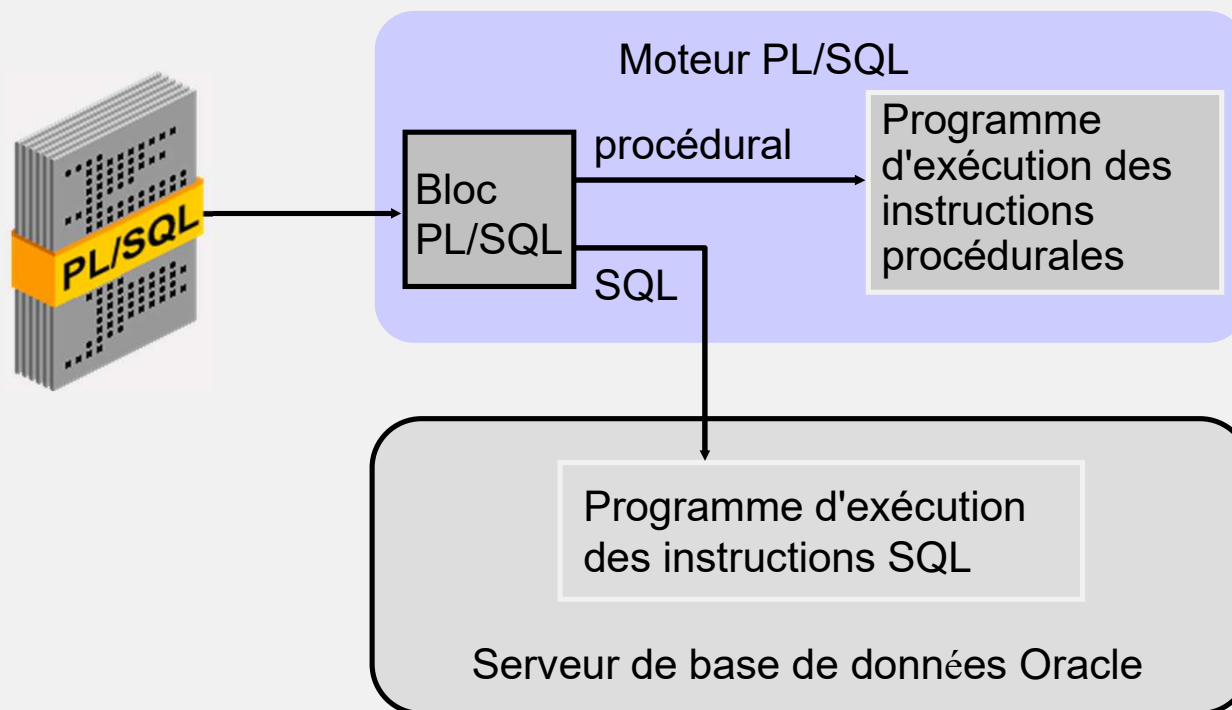


Structure du bloc PL/SQL (3/11)

Le langage PL/SQL :

- Offre une structure de bloc pour les unités de code exécutables. La maintenance du code est facilitée avec une structure bien définie.
- Fournit des structures procédurales telles que :
 - *Variables, constantes et types*
 - *Structures de contrôle, telles que les instructions conditionnelles et les boucles*
 - *Programmes réutilisables écrits une fois et exécutés plusieurs fois*

Structure du bloc PL/SQL (4/11)



Structure du bloc PL/SQL (5/11)

- Un bloc PL/SQL contient des instructions procédurales et des instructions SQL.
- Si on soumet un bloc PL/SQL au serveur, le moteur PL/SQL commence par analyser le bloc : Il identifie les instructions procédurales et les instructions SQL.
- Il transmet les instructions procédurales au programme d'exécution des instructions procédurales et transmet les instructions SQL au programme d'exécution des instructions SQL.
- Par conséquent, toutes les instructions procédurales sont exécutées localement et seules les instructions SQL sont exécutées dans la base de données.

Structure du bloc PL/SQL (6/11)

La structure de base d'un programme PL/SQL est celle de bloc (possiblement imbriqué)

La délimitation des blocs est faite avec les mots réservés:

DECLARE (facultatif)

Variables, curseurs, exceptions définies par l'utilisateur

BEGIN (obligatoire)

- Instructions SQL
- Instructions PL/SQL

EXCEPTION (facultatif)

Actions à effectuer lorsque des erreurs se produisent

END; (obligatoire)



Structure du bloc PL/SQL (7/11)

- Le corps du programme (entre le BEGIN et le END) contient des instructions PL/SQL (assignements, boucles, appel de procédure) ainsi que des instructions SQL.
- Il s'agit de la seule partie qui soit obligatoire. Les deux autres zones, dites zone de déclaration et zone de gestion des exceptions sont facultatives.
- Les seuls ordres SQL que l'on peut trouver dans un bloc PL/SQL sont: SELECT, INSERT, UPDATE, DELETE.
- Les autres types d'instructions (par exemple CREATE, DROP, ALTER) ne peuvent se trouver qu'à l'extérieur d'un tel bloc. Chaque instruction se termine par un ";".
- Le PL/SQL ne se soucie pas de la casse. On peut inclure des commentaires par -- (en début de chaque ligne commentée) ou par /*.... */ (pour délimiter des blocs de commentaires).

Structure du bloc PL/SQL (8/11)

Ainsi :

- La partie de déclaration est optionnelle, mais chaque objet utilisé doit être déclaré: Variables, Constantes, Types, Curseurs, ...
- La partie des commandes exécutables est toujours présente. Elle est constituée de :
 - *Ordres SQL*
 - *Manipulations des variables, constantes, et de structures de programmation (itérations, sélections, etc.).*
- La partie des exceptions est optionnelle
 - *Elle gère les exceptions et les reprises d'erreurs.*

Structure du bloc PL/SQL (9/11)

Premiers pas en pl/sql

Programme1

Begin

End;

Programme2

Begin

End;

/

```
SQL> Begin
 2 End;
 3
 4
 5
 6
 7
 8 |
```

Un bloc pl/sql doit se terminer par un /

Un bloc pl/sql doit contenir au moins une instruction

```
SQL> begin
 2 end;
 3 /
end;
*
ERREUR à la ligne 2 :
ORA-06550: Ligne 2, colonne 1 :
PLS-00103: Symbole "END" rencontré à la place d'un des symboles suivants :
begin case declare exit for goto if loop mod null pragma
raise return select update while with <identificateur>
<identificateur entre guillemets> <variable bind> << close
current delete fetch lock insert open rollback savepoint set
sql execute commit forall merge pipe
```

Structure du bloc PL/SQL (10/11)

Programme 3

Declare

Mot varchar2(20);

End;

/

Le bloc d'exécution
Begin/End est
obligatoire

```
SQL> Declare
      2 Mot varchar2(20);
      3 End;
      4 /
End;
*
ERREUR à la ligne 3 :
ORA-06550: Ligne 3, colonne 1 :
PLS-00103: Symbole "END" rencontré à la place d'un des symboles suivants :
begin function package pragma procedure subtype type use
<identificateur> <identificateur entre guillemets> form
current cursor
```

Structure du bloc PL/SQL (11/11)

Programme 4 : Affichage

Set serveroutput on

Declare

Chaine varchar2(10) := ' Bonjour ';

Begin

Dbms_output.put_line (chaine);

End;

/

Permet l'activation de l'affichage sur ecran

```
SQL> set serveroutput on
SQL>
```

Toute variable doit avoir été déclarée avant de pouvoir être utilisée dans la section exécutable.

```
SQL> Declare
2  Chaine varchar2(10) := ' Bonjour ';
3  Begin
4  Dbms_output.put_line (chaine);
5  End;
6  /
Bonjour
```

Procédure PL/SQL terminée avec succès.

Les variables (1/17)

La section déclarative

Syntaxe :

nom variable [CONSTANT] type [[NOT NULL] := expression] ;

- **nom variable** représente le nom de la variable composé de lettres, chiffres, \$, # ou _
Le nom de la variable ne peut pas excéder 30 caractères, doit commencer par une lettre, n'est pas sensible à la casse et doit être déclarée avant d'être utilisée.
- **CONSTANT** indique que la valeur ne pourra pas être modifiée dans le code du bloc PL/SQL
- **NOT NULL** indique que la variable ne peut pas être NULL, et dans ce cas **expression** doit être indiqué.
- **TYPE** représente le type de la variable (→)

Les variables (2/17)

(→)

Plusieurs types de variables sont manipulés par un programme PL/SQL :

Variables PL/SQL :

- Types scalaires recevant une seule valeur :

INTEGER, REAL, STRING, DATE, BOOLEAN + %TYPE + types SQL (Tous les types SQL sont utilisables en PL/SQL)

- Types composites (%ROWTYPE, RECORD)

Variables non PL/SQL :

- définies sous SQL*Plus (de substitution et globales), Variables hôtes (déclarées dans des programmes précompilés).

Les variables (3/17)

Exemples

- *age integer;*
- *nom varchar2(30);*
- *dateNaissance date;*
- *ok boolean := true;*

Rq: Déclarations multiples **interdites** :

~~Exemple: *i, j integer;*~~

Les variables (4/17)

Exemple

```
declare
date_naissance DATE := '03/02/2008';
compteur integer default 0;
id char(5) not null ;
begin
dbms_output.put_line(date_naissance||' '||compteur||' '||id);
end;
/
```

```
ERREUR à la ligne 4 :
ORA-06550: Ligne 4, colonne 4 :
PLS-00218: une variable déclarée NOT NULL doit avoir une affectation
d'initialisation
```

Les variables (5/17)

Le type %Type

- référence à un type existant qui est soit une colonne d'une table (ou d'une vue) soit un type défini précédemment

nom_variable nom_table.nom_colonne%TYPE ;

nom_variable nom_variable_ref%TYPE ;

Exemple1

Declare

idProjet projet.numproj%Type;

Exemple2

Declare

Date1 DATE;

Date2 Date1%Type;

Les variables (6/17)

Le type %RowType

- Ce type est composé d'un ensemble de colonnes d'un enregistrement. L'enregistrement peut contenir toutes les colonnes d'une table ou seulement certaines.
- Il fait référence à une ligne d'une table

– Exemple

Declare

Employe personne%Rowtype;

– Utilité:

- diminue les changements à apporter au code en cas de modification des types des colonnes de la table.
- Il est aussi possible d'insérer dans une table ou de modifier une table en utilisant une variable de type %ROWTYPE.

Les variables (7/17)

Exercice

- *Créer le type un service composé d'un enregistrement de la table service et dont tous les composants sont de même type que la table service avec la valeur du numéro accordé au service étant choisie par défaut et les autres valeurs quelconques.*
- *Insérer le service crée au niveau de la table service puis le visualiser.*

Les variables (8/17)

Le type Record : permet de déclarer des structures de données personnalisées.

```
TYPE nomRecord IS RECORD  
  ( nomChamp1 typeDonnées [[NOT NULL] {:= | DEFAULT} expression]  
    [,nomChamp2 typeDonnées... ]... );
```

– Exemple

```
DECLARE  
TYPE employe IS RECORD  
  (numemp NUMBER(4),  
   Nomemp VARCHAR2(16),  
   postemp VARCHAR2(12) := 'ingénieur');  
Emp1 employe;
```

Déclaration du RECORD employe contenant trois champs ;
initialisation du champ postemp par défaut à ingénieur.

Déclaration d'une variable de type employe

Les variables (9/17)

Remarque il est possible qu'un champ d'un RECORD soit lui-même un RECORD, ou soit déclaré avec les directives %TYPE ou %ROWTYPE.

Exemple

– *Déclarer une structure « point » composée d'une abscisse et d'une ordonnée. Afficher l'abscisse et l'ordonnée d'un point de votre choix.*

Les variables (10/17)

Affectation

Il existe plusieurs façons de donner une valeur à une variable :

- `:=`
- *Par la directive default*
- *Par la directive INTO de la requête SELECT*

Exemples :

- `age := 25;`
- `Afficher en utilisant la variable nompersonne la personne ayant le numéro 7501`

Les variables (11/17)

Le type Varray

TYPE *nomTypeTableau* IS VARRAY (taille) OF typeElements ;

NomTableau nomTypeTableau;

Les variables (12/17)

Le type Table

TYPE *nomTypeTableau* IS TABLE OF

{typeScalaire | variable%TYPE | table.colonne%TYPE} [NOT NULL]

| table.%ROWTYPE [INDEX BY BINARY_INTEGER];

nomTableau nomTypeTableau;

- Permet de définir et manipuler des tableaux dynamiques (car définis sans dimension initiale).
- Un tableau est composé d'une clé primaire (de type `BINARY_INTEGER`) et d'une colonne (de type scalaire, `TYPE`, `ROWTYPE` ou `RECORD`) pour stocker chaque élément.
- La plage de valeurs du type `BINARY_INTEGER` est comprise entre -2 147 483 647 et 2 147 483 647, ce qui signifie que la valeur de la clé primaire peut être négative. L'indexation ne doit pas nécessairement commencer à 1.

Les variables (13/17)

Fonctions pour les tableaux

- *EXISTS(x)* Retourne *TRUE* si le $x^{\text{ème}}$ élément du tableau existe.
- *COUNT* Retourne le nombre d'éléments du tableau.
- *FIRST / LAST* Retourne le premier/dernier indice du tableau (*NULL* si tableau vide).
- *PRIOR(x) / NEXT(x)* Retourne l'élément avant/après le $x^{\text{ème}}$ élément du tableau.
- *DELETE; DELETE(x); DELETE(x,y)* Supprime un ou plusieurs éléments du tableau.

Les variables (14/17)

Exercice

- *Déclarer un tableau dynamique de chaîne de caractères dont les indices -3, -2, 0 contiennent des noms de personnes ayant les numéros 7369,7566,7901.*
- *Afficher le premier indice, le dernier indice ainsi que le nombre d'éléments du tableau.*
- *Afficher le contenu du tableau correspondant à l'indice 0.*

Les variables (15/17)

Variables non pl/sql

Variables de substitution

- *Il est possible de passer en paramètres d'entrée d'un bloc PL/SQL des variables définies sous SQL*Plus.*
- *Ces variables sont dites de substitution.*
- *On accède aux valeurs d'une telle variable dans le code PL/SQL en faisant préfixer le nom de la variable du symbole « & » (avec ou sans guillemets simples suivant qu'il s'agit d'un nombre ou pas).*
- **Syntaxe**

***ACCEPT variable [NUMBER | CHAR | DATE | BINARY_FLOAT |
BINARY_DOUBLE] [PROMPT text | NOPROMPT***

Les variables (16/17)

Exemple

Ecrire un programme qui permet de demander la saisie d'un numéro puis de l'afficher.

Les variables (17/17)

Variables de session

- *Il est possible de définir des variables de session (globales) définies sous SQL*Plus au niveau d'un bloc PL/SQL.*
- *La directive SQL*Plus à utiliser en dehors du bloc PL/SQL est **VARIABLE**.*
- *Dans le code PL/SQL, il faut faire préfixer le nom de la variable de session du symbole « : ».*
- *L'affichage de la variable sous SQL*Plus est réalisé par la directive PRINT.*

Exemple

- *Déclarer une variable globale var1*
- *Affecter a cette variable la somme d'un nombre num + 5*
- *Afficher cette variable*

Traitement conditionnel (1/5)

IF...
THEN...
END IF;

IF...
THEN...
ELSE...
END IF;

IF...
THEN...
ELSIF...
THEN...
END IF;

IF...
THEN...
ELSIF...
THEN...
ELSE...
END IF;

CASE
WHEN... THEN..
WHEN... THEN..
WHEN... THEN..
ELSE
END CASE;

Traitement conditionnel (2/5)

Syntaxe de la structure IF :

```
IF <condition> THEN commandes;  
[ ELSIF <condition> THEN commandes; ]  
[ ELSE commandes; ]  
END IF;
```

Exemple

Ecrire un programme qui permet d'afficher qu'une personne, selon son âge, est un enfant (avant 13 ans) est un ado (entre 13 et 18) ou un adulte (après 18 ans).

Traitement conditionnel (3/5)

Syntaxe de la structure CASE

```
CASE [variable]  
WHEN expr1 THEN instructions1;  
WHEN expr2 THEN instructions2;  
....  
[ELSE instruction N;]  
END CASE ;
```

Traitement conditionnel (4/5)

Exemple

Donner en fonction de la note obtenue d'un étudiant la mention :

- *Très bien si la note est > 16*
- *Bien si la note est entre 14 et 16*
- *Assez bien si la note est entre 12 et 14*
- *Passable si la note est entre 10 et 12*

Traitement conditionnel (5/5)

Exercice

Ecrire un programme permettant de dire pour les personnes:

- 7501 dans quel numéro de service elle est affectée
- 7901 combien de personnes travaillent avec elle dans ce service.
- 7902 la moyenne des salaires de son service.